Van Stock & Dispatch

Seven vans, one spreadsheet, and a restaurant that opened four hours late because of a printer that was in the van all along. Build us the system that fixes it.

Build challenge · Hearthline · issued 2026-10-14

What Hearthline does

Hearthline supplies and services the hardware restaurants run on: tills, receipt and kitchen printers, card readers, routers. Each restaurant is a **Customer** with one or more **Branches**. **Technicians** drive **Vans** that carry **Stock** — units in boxes, spares, cables, paper — issued from one **Hub**, our warehouse, forty minutes from most of the city. Serialised hardware is a **Unit** with a serial number. Visits are **Deliveries**, **Installations** and **Repairs**; a repair swaps the dead unit for a working one and brings the dead one back.

The problem

There is no system. The team runs on Google Sheets and WhatsApp. Dispatch books visits and arranges deliveries in the sheet you have been sent, technicians report back in a group chat, and the hub counts shelves on Mondays. It is slow and it is full of human error. A van is picked because the sheet says it has the parts, and the technician arrives to find them dead, missing, or in another van. A request sits in the sheet until someone notices it. Nobody can say what a van is carrying right now, or which of tomorrow's jobs can actually be done with what is on the road.

Our promise to customers is **delivery and installation the next working day, anywhere in the city.** The sheet says we currently take **almost a week.**

> **What we are asking you to build**
>
> The first system this team has had. Not a fix or an optimisation of something that exists, because nothing does: requests, scheduling, what is on each van, and restocking in one place, so that a van with the right parts reaches the restaurant the next working day, with no spreadsheet in the loop.

What is in the pack

Document	What it is
This brief	Why we are doing this, and what has to exist when you are done.
Email thread `02-email-thread.pdf`	An argument between Dispatch, the Field supervisor and the Hub about what is on the vans. You have been copied in. Most of the real requirements are in here.
Operations board `03-ops-board.pdf`	The Field supervisor's whiteboard: how the process is supposed to run.
Van stock book `04-van-stock-book.xlsx`	An export of the Google Sheet the operation runs on: every request since August and when we actually went, what is on each van, the agreed minimums. The closest thing we have to a database.

> **These are working documents, not a specification**
>
> They were written by different people for different reasons. They are not consistent with each other, and in places they are silent. **Where they disagree or say nothing, make a call and write down what you decided and why.** That record is part of what we read, and a well-argued decision we would have made differently costs you nothing.

What has to exist when you are done

Van Stock & Dispatch · case brief · Hearthline build challenge · 2026-10-14

Three things, whatever else you choose to build. These are the parts we compare across candidates. You are starting from nothing, so the shape of the whole thing is yours to decide.

	Requirement
1	**The book, loaded.** The stock book and the job log in a database. Be explicit, and visible, about the rows that did not fit.
2	**The dispatch board.** Every open request and how long it has waited; every van, what it carries, who is driving it, what it is doing today; and the means to put a job on a van that can actually do it.
3	**The restock list.** When a van falls below its minimum, a restock job with a pick list: what to collect and how many. One list, produced one way.

If you have time left, the next thing we would build is the technician's screen: today's jobs on a phone, close a job, record what went in and what came out, and the van's stock moves. Beyond that, build what you think the operation needs; we care more about what you chose and why than about how much you finished.

How to spend the time

About four hours: **one with the material, three building.** The reading is not overhead — the requirements are in those documents. You have a 48-hour window to fit this around your job; please do not give it a weekend, and spending longer does not improve your score. Unfinished is normal. What we cannot evaluate is work you did not make a decision about.

Stack: yours. We work in React/TypeScript, Node/Express and PostgreSQL, but build in whatever you are fastest in; it is not scored. **AI tools:** use whatever you use at work. This is expected, not tolerated. We will ask how you used them, and we will read the session history you send us.

What to submit

	Item
1	A repository (link or zip), plus a running version if it is easy to deploy. If it is not easy, do not bother.
2	A 3–5 minute screen recording of what you built. We watch it before the call.
3	**Your commit history, intact.** Do not squash.
4	**Your AI session history** — chat exports, prompt logs, whatever your tool produces. **Turn this on before you start;** most tools discard it by default.
5	A README, one page: what you built, what you cut, what you would do next, and the decisions you made where the material did not tell you what to do.

What happens on the call

Two of us read your submission before we meet. On the call you share your screen for 90 minutes. We open files we picked in advance and ask how they work and why — including the parts you did not type yourself — and we ask what you would not ship. Then we hand you two new requirements and you build them live, in your own tools. **Have the project running before the call starts.** Something will break; that is part of the exercise.

Questions are not a penalty. If something in the pack stops you dead, ask us. Asking and assuming are both legitimate; we only want to know which one you did, and why.

Hearthline is a company invented for this exercise. The people, customers and problem are fictional, and nothing you build will be used by us.